RAMCO INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Academic Year: 2019- 2020 (Odd Semester)

Innovative practice(s) description

Course code and title: CS8592 Object Oriented Analysis and Design
Name of the Instructor(s): Mr.D.Lakshmanan, AP(S.G)/CSE
Date & Time: 30.09.2019 & 9.50AM to 10.40AM

Innovative Practice: Simulation
Topic: Simulation on unit testing

Objectives:

- To make the students to test the java POJO class using JUNIT jar.
- To make the students to apply the unit testing on java class.

Reason for choosing the particular topic:

JUnit is a unit testing framework for Java programming language. It plays a crucial role test-driven development, and is a family of unit testing frameworks collectively known as xUnit.

JUnit promotes the idea of "first testing then coding", which emphasizes on setting up the test data for a piece of code that can be tested first and then implemented. This approach is like "test a little, code a little" It increases the productivity of the programmer and the stability of program code, which in turn reduces the stress on the programmer and the time spent on debugging.

I used eclipse IDE for installing Junit jar. Eclipse is a famous tool for creating Java EE and Web applications.

Activity:

During the second hour on 30.09.2019 Simulation on unit testing innovative practice conducted for III year CSE students, the Junit jar is used to test the java POJO class as below.

In the class I will explain one complete example of JUnit testing using POJO class, Business logic class, and a test class TestEmployeeDetails, which will be run by the TestRunner class as follows.

Instructions to perform a simulation experiment.

Open Eclipse Ide.

Create new Java project

Set Junit Jar in build path as follow

1. Right click on your project.
2. Select Build Path.
3. Click on **Configure** Build Path.
4. Click on Libraries and select Add External **JARs**.

5. Select the **jar file** from the required folder.
6. Click and Apply and Ok

Then create following classes in your project as follow

Create **EmployeeDetails.java (POJO CLASS)**

```
public class EmployeeDetails {

	private String name;
	private double monthlySalary;
	private int age;

	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public double getMonthlySalary() {
		return monthlySalary;
	}
	public void setMonthlySalary(double monthlySalary) {
		this.monthlySalary = monthlySalary;
	}
	public int getAge() {
		return age;
	}
	public void setAge(int age) {
		this.age = age;
	}
}
```

Then Create Business Logic Class as follows

```
public class EmpBusinessLogic {
  // Calculate the yearly salary of employee
  public double calculateYearlySalary(EmployeeDetails employeeDetails) {
    double yearlySalary = 0;
    yearlySalary = employeeDetails.getMonthlySalary() * 12;
    return yearlySalary;
  }

  // Calculate the appraisal amount of employee
  public double calculateAppraisal(EmployeeDetails employeeDetails) {
    double appraisal = 0;

    if(employeeDetails.getMonthlySalary() < 10000){
      appraisal = 500;
    }else{
      appraisal = 1000;
    }
```

```
    return appraisal;
  }
}
```

Then Create TestEmployeeDetails class as follow

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class TestEmployeeDetails {
  EmpBusinessLogic empBusinessLogic = new EmpBusinessLogic();
  EmployeeDetails employee = new EmployeeDetails();

  //test to check appraisal
  @Test
  public void testCalculateAppriasal() {
    employee.setName("Rajeev");
    employee.setAge(25);
    employee.setMonthlySalary(8000);

    double appraisal = empBusinessLogic.calculateAppraisal(employee);
    assertEquals(500, appraisal, 0.0);
  }

  // test to check yearly salary
  @Test
  public void testCalculateYearlySalary() {
    employee.setName("Rajeev");
    employee.setAge(25);
    employee.setMonthlySalary(8000);

    double salary = empBusinessLogic.calculateYearlySalary(employee);
    assertEquals(96000, salary, 0.0);
  }
}
```

Then Create TestRunner class as follows

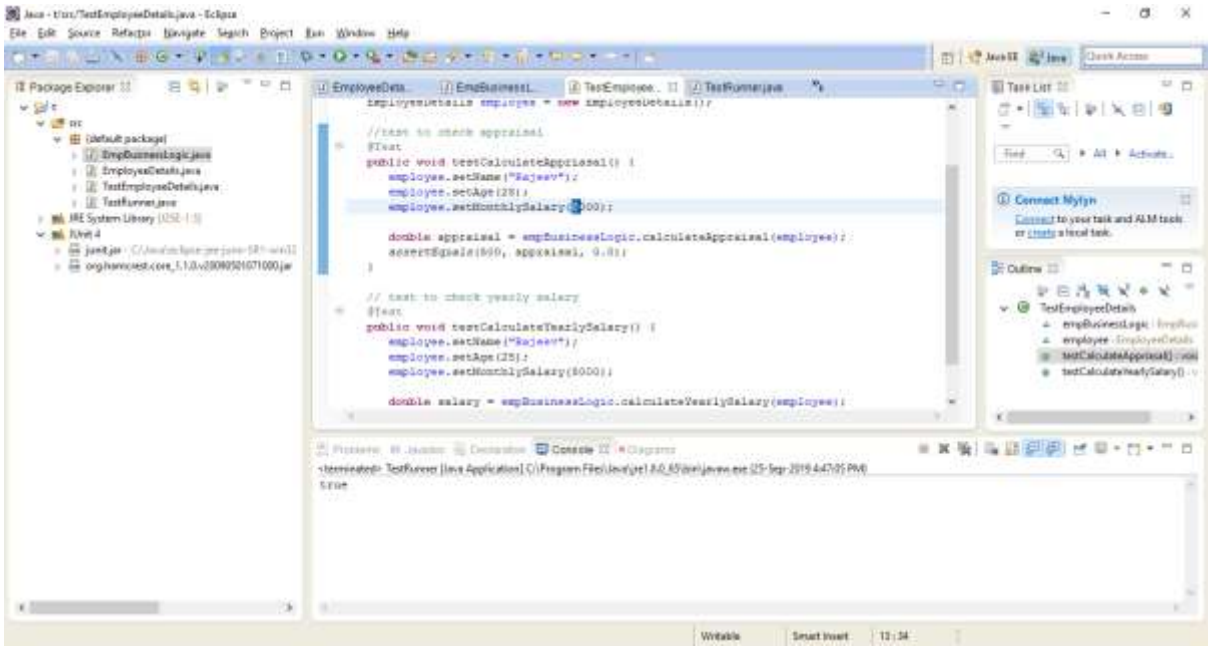
```
import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

public class TestRunner {
        public static void main(String[] args) {
    Result result = JUnitCore.runClasses(TestEmployeeDetails.class);

    for (Failure failure : result.getFailures()) {
      System.out.println(failure.toString());
    }

    System.out.println(result.wasSuccessful());
  }
}
```

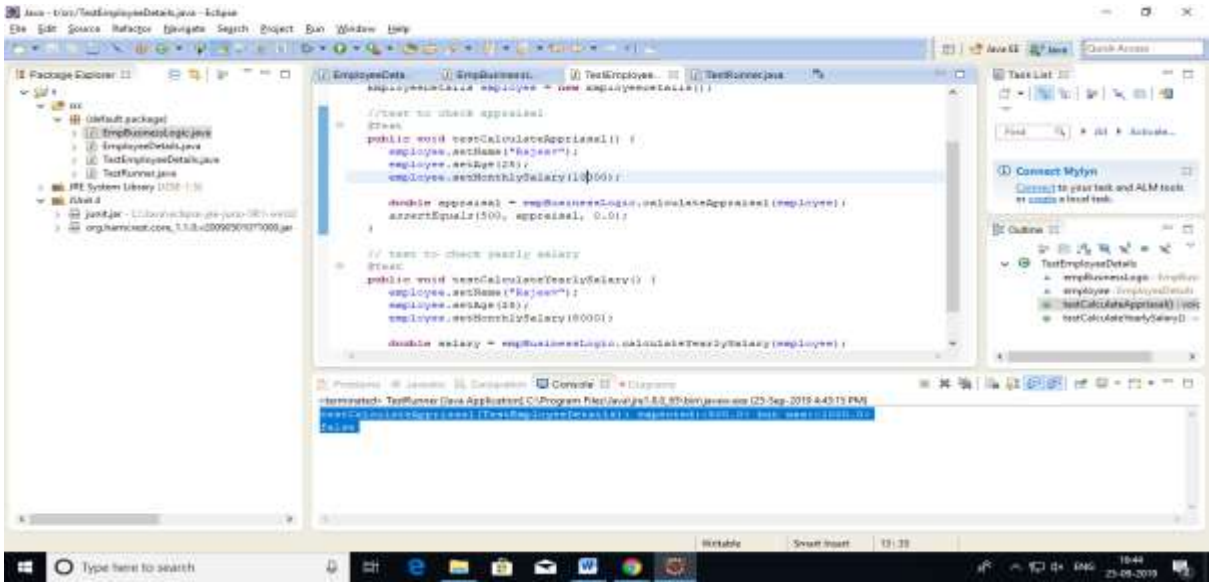
Two test cases are created in the TestEmployeeDetails as follows

1. **<u>testCalculateAppriasal()</u>**

   To check the appraisal value equal to 500 when the salary is less than 10000

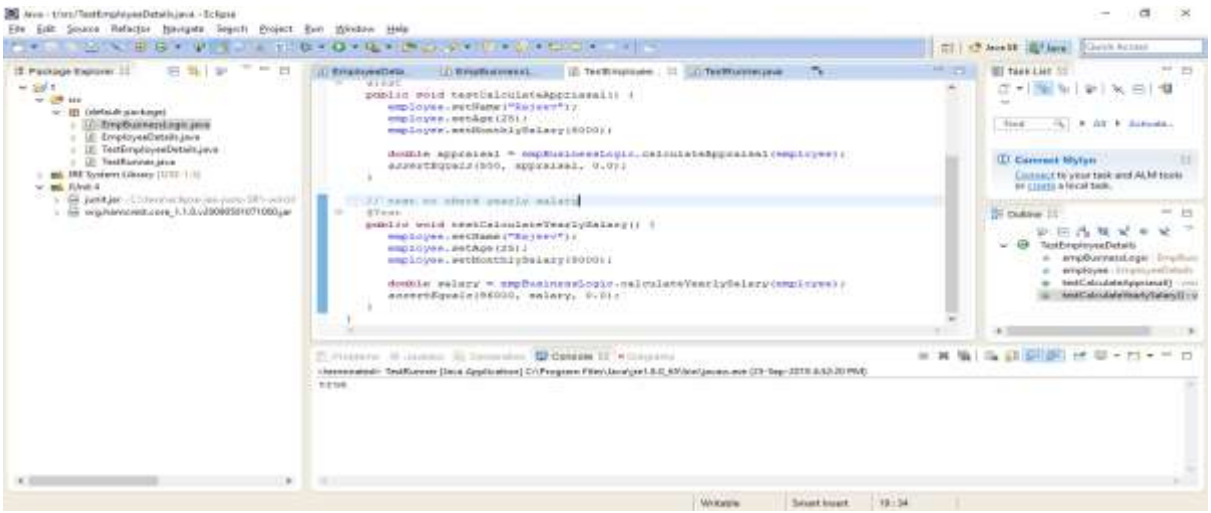
   assertEquals(500, appraisal, 0.0)
   returns True when the salary is less than 10000.
   returns False when the salary is greater than 10000

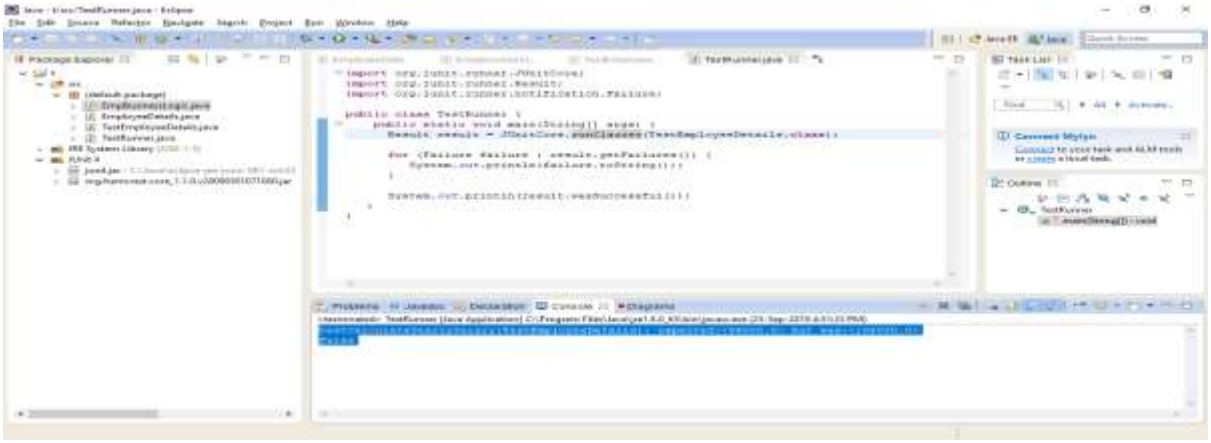




2. **testCalculateYearlySalary()**

   In the testCalculateYearlySalary() test case, it multiply the salary with 12 and check the answer is correct or not using assertEquals() method as follow.





Finally, to test the test case run the TestRunner class as above. It will show test case result is true or false as above.

Frequent Ask Questions and answers

What is the need of unit testing?

Unit testing improves the quality of the code. It identifies every defect that may have come up before code is sent further for integration testing. Writing tests before actual coding makes you think harder about the problem. It exposes the edge cases and makes you write better code.

This activity helps in attaining the following CO – PO mapping:

Course Outcome / Programme Outcome	PO1	PO2	PO3	PO5
Transform UML based software design into pattern based design using design patterns.	3	2	2	3

SESSION OUTCOMES:

- This makes the students to test the java class using JUnit.
- It provides opportunities for the students to acquire knowledge about unit testing and test driven development.

Reflection Critique

Challenges:

- Slow learners were not able to understand the real time testing happening in industry.
- Some of the students unable to understand what is unit testing.
- Some of the students not understand what POJO class is.

Steps taken for address the challenges:

I used JUNIT jar file to test the java class and display the testing result in console. Test cases are checked with assertEquals method. It shows whether the test cases are pass or fail.

So students come to know what is unit testing and POJO(Plain Old Java Object) classes. They can able to realize the real time testing happening in industry.

Reference

1.https://www.tutorialspoint.com/junit/index.htm

Prepared by: **Approved by:**

Mr. D.Lakshmanan, AP(S.G)/CSE **Dr. K.Vijayalakshmi, HOD/CSE**